

Abstract

This manuscript reports a paired empirical comparison of three context-delivery treatments for Lattice Chat: a pointer-grounded nested prompt (**lattice**), a lean direct prompt (**standard**), and a bounded repository-corpus dump (**naive**). The comparison measures provider-reported token usage, request latency, deterministic task accuracy, lenient answer containment, and paired treatment differences. The primary completed three-treatment run used **deepseek/deepseek-chat**, 3 repeats, 9 tasks, and alternating treatment order, yielding 27 observations per treatment. Lattice accuracy was 0.593, standard accuracy was 0.407, and naive accuracy was 0.444; mean total tokens were approximately 2,999, 1,036, and 22,651, respectively. Thus Lattice used approximately 0.132 times the naive tokens, or 86.759% fewer tokens, while exceeding its accuracy in this descriptive run. A second model run with **openai/gpt-4.1** yielded lattice accuracy 0.556 versus standard 0.333, paired two-sided $p = 0.042$ and Cohen's $d_z = 0.519$, but used only 2 repeats. The observed advantage is concentrated in repository questions whose answers are present in the Lattice grounding envelope; arithmetic reasoning was essentially equivalent across treatments. These results support a narrower claim: explicit repository grounding can improve task performance relative to a lean prompt, while a pointer envelope can be substantially cheaper than a large corpus dump. They do not establish that nesting itself is causally responsible, nor do they justify generalization beyond the tested models, tasks, and small samples.

Keywords: context engineering; retrieval grounding; nested agents; token efficiency; paired experiment; OpenRouter; reproducibility

Honesty boundary. Provider usage is measured for the recorded calls. The experiment is not a causal isolation of nesting, retrieval, or model reasoning; the task battery and prompt envelopes jointly define the treatments. Small repeat counts make all inferential statistics provisional.

Contributions

1. A reproducible treatment definition separating lean, pointer-grounded, and corpus-dump context.
2. A deterministic battery with strict and lenient scoring paths.
3. Paired token, latency, accuracy, confidence, and effect-size summaries.
4. An auditable record of an invalid OpenRouter model identifier rather than silently dropping it.
5. Self-contained figures and modular source suitable for the template rendering pipeline.

Manuscript map

The introduction gives background and claims boundaries. The formalism defines context budgets and paired outcomes. Methods specify the protocol and

scoring. Results report the completed runs and the failed model-ID audit. Discussion separates mechanism, implementation, applicability, and limitations. Reproducibility binds every claim to repository artifacts.

Data availability

The raw response text is intentionally excluded from committed result JSON and reports. Aggregated JSON/CSV records, generated HTML reports, source code, and this manuscript are available in the repository paths listed in [sec:reproducibility]. API credentials are not part of the artifact set.

Introduction

Context as an experimental variable

A language-model request is not determined only by the user question. It is a composition of a user message, system instructions, repository context, retrieved pointers, tool affordances, and the model’s response budget. In a repository agent, these components can be delivered as a compact set of navigational pointers, as a lean direct prompt, or as a large context dump. The practical question is not whether more context is always better, but whether context can be selected and structured so that it improves task performance without making every request pay the cost of the whole corpus.

Lattice Chat operationalizes one answer: a nested prompt envelope identifies a parent execution frame, Seed/Edge/Pipes bands, repository pointers, and mode-specific directives. The envelope is intended to make relevant structure legible while preserving scale-to-zero behavior. The comparison in this manuscript treats the entire envelope as the intervention. It therefore tests the experience of “Lattice context” rather than isolating any single internal mechanism.

Research questions

RQ1 — Token economy. How do provider-reported tokens differ between a pointer-grounded Lattice envelope, a lean direct prompt, and a bounded corpus dump?

RQ2 — Task performance. Does explicit repository grounding change accuracy on repository QA, deterministic reasoning, and a patch-generation task?

RQ3 — Mechanism. Are any observed differences consistent with answer availability in the prompt envelope rather than with a general reasoning advantage?

RQ4 — Robustness. Do the direction and magnitude of effects appear across more than one valid OpenRouter model?

Claims and non-claims

The supported claim is operational and bounded: in the recorded runs, a pointer-grounded prompt consumed far fewer tokens than the naive corpus-dump baseline, and it sometimes improved repository-answer accuracy relative to a lean prompt. The manuscript does not claim that a nested-agent architecture is universally superior, that token reduction implies equal quality, or that the measured p-value survives correction for task selection, model selection, or repeated exploration.

```

flowchart LR
    U[User task] --> O[Experiment orchestrator]
    O --> P{Treatment}
    P --> L["Lattice envelope<br/>Seed/Edge/Pipes pointers"]
    P --> S["Standard<br/>lean direct prompt"]
    P --> N["Naive<br/>bounded corpus dump"]
    L --> M[OpenRouter model]
    S --> M
    N --> M
    M --> R[Response + provider usage]
    R --> Q[Deterministic scorer]
    Q --> T[Paired statistics]
    T --> V[JSON/CSV + SVG/HTML + manuscript]

```

Figure 1: Mermaid diagram

System architecture

Figure interpretation. The three arms share the task, model, temperature, repetition schedule, and scoring layer. Context construction is the treatment boundary. The orchestrator records order, latency, usage, and scores; the report generator removes raw response text from committed artifacts.

Related framing

The experiment sits at the intersection of retrieval-augmented generation, context-window management, agent orchestration, and software-engineering evaluation. Its distinctive constraint is not merely retrieval quality: it asks whether a compact architectural description can act as a stable routing layer across heterogeneous repository tasks. The appropriate comparison is therefore multi-objective: accuracy, token cost, latency, and failure mode must be shown together.

Formalism

Context-treatment model

Let x_i denote task i , m a model, r a repeat, and t a treatment. A request is represented as

$$q_{i,m,r,t} = (u_i, s_t, c_t, \tau, b),$$

{#eq:request}

where u_i is the task prompt, s_t is the treatment system instruction, c_t is treatment-specific context, $\tau = 0$ is the sampling temperature, and b is the maximum completion budget. The response is

$$y_{i,m,r,t} \sim M_m(q_{i,m,r,t}).$$

{#eq:response}

The experiment compares the joint effect of (s_t, c_t) ; it does not identify a structural causal effect of any subcomponent.

Treatments

The standard treatment uses a short direct system instruction and no repository corpus for QA/reasoning. For coding, it receives a bounded snapshot of three current files. The Lattice treatment uses `assembleLatticePrompt` with Goldilocks topology and the same task snapshot as a bounded user-context supplement. The naive treatment concatenates a deterministic, size-ranked set of repository files up to a fixed character budget of 70,000 characters. This operationalizes a fat-context baseline without claiming to reproduce every possible corpus dump.

Let C_t be input characters and $\kappa = 4$ the structural chars-per-token approximation. The structural context estimate is

$$\widehat{K}_t = \left\lceil \frac{C_t}{\kappa} \right\rceil.$$

{#eq:structural_tokens}

Provider-reported total tokens $K_{i,m,r,t}$ are used for empirical token results whenever available. Structural estimates are never substituted for provider usage in the live-run tables.

Outcomes

For task correctness, $A_{i,m,r,t} \in \{0,1\}$ is the deterministic strict score. The lenient score $A_{i,m,r,t}^\ell$ accepts a repository answer when the expected token occurs

in the response and extracts the first numeric answer for reasoning tasks. Coding correctness requires a returned unified diff to apply cleanly and the resulting temporary checkout to pass `npm ci` and `npm test -- --run`.

Latency is $L_{i,m,r,t}$ in milliseconds. Token efficiency is reported as total tokens per correct outcome; when an individual response is incorrect, its token count remains visible and is not deleted. Aggregate `tokensPerCorrect` is therefore an operational burden metric, not a probability estimator.

Paired contrasts

For two treatments a and b , the within-pair difference for outcome z is

$$\Delta_{i,m,r}^{(z)} = z_{i,m,r,a} - z_{i,m,r,b}.$$

{#eq:paired_difference}

The paired mean difference is

$$\bar{\Delta}^{(z)} = \frac{1}{n} \sum_{j=1}^n \Delta_j^{(z)},$$

{#eq:mean_difference}

with standard error $SE(\bar{\Delta}) = s_{\Delta}/\sqrt{n}$. The report includes a two-sided paired t statistic, a Wilcoxon signed-rank statistic where defined, a 95% confidence interval based on the t critical value, and Cohen’s paired effect size

$$d_z = \frac{\bar{\Delta}}{s_{\Delta}}.$$

{#eq:cohens_dz}

These inferential summaries are descriptive because task observations are heterogeneous and the repeat counts are small.

Alternating order

To reduce a simple time/order confound, repeat r uses treatment order a, b for even-indexed repeats and b, a for odd-indexed repeats. Pair identity is the tuple (i, m, r) , never the visual row order in a report.

Information-flow interpretation

The Lattice envelope can improve a repository QA answer through a direct grounding path:

flowchart TD

```
A[Repository fact] --> B[Pointer/catalog or directive]
B --> C[Lattice system envelope]
C --> D[Model attention]
D --> E[Answer]
F[Lean standard prompt] --> D2[Model attention without fact]
D2 --> G[Answer from model recall/inference]
```

Figure 2: Mermaid diagram

This is a testable mechanism hypothesis. It predicts the largest advantage on questions whose exact answer is surfaced by the Lattice envelope, not necessarily on closed-form arithmetic.

Threat model for interpretation

The primary threats are prompt leakage (answer availability differs), unequal context budgets, model-specific instruction-following, strict scoring mismatch, coding-harness failure, multiple exploratory runs, and invalid model identifiers. The protocol records these rather than collapsing them into a single headline.

Methods

Repository and implementation

The experiment implementation is dependency-free ESM in `lib/openrouter-experiment.mjs` and is orchestrated by `scripts/lattice-vs-standard-openrouter-usage.mjs`. The runner loads environment configuration without writing credentials, constructs treatment messages, calls OpenRouter, records provider usage and latency, scores responses, computes paired summaries, and writes JSON, CSV, and self-contained HTML/SVG reports.

Task battery

Class	ID	Operational target
QA	<code>qa_default_cursor_model</code>	Recover the default Cursor model identifier
QA	<code>qa_cli_email_env</code>	Recover the CLI email environment variable
QA	<code>qa_cli_default_nest</code>	Recover the default <code>nestTopology</code> value
QA	<code>qa_openrouter_header</code>	Recover the OpenRouter BYOK header
QA	<code>qa_cli_auth_exit_code</code>	Recover the auth/access error exit code
Coding	<code>coding_add_top_p_flag</code>	Return an applicable CLI patch that passes the test suite
Reasoning	<code>reasoning_arithmetic</code>	Compute 17×43
Reasoning	<code>reasoning_remainder</code>	Compute $1024 \bmod 7$
Reasoning	<code>reasoning_sequence</code>	Compute the sixth term of $n(n + 1)$

QA tasks were originally requested as answer-only responses and are scored strictly by normalized exact match. The report also includes a lenient containment score to diagnose whether strict scoring is rejecting prose answers. Reasoning extracts a numeric answer. Coding is an end-to-end patch gate, not a code-quality rating.

Live runs

The planned protocol is nine tasks $\times$ treatments $\times$ repeats. The completed valid datasets were:

Dataset	Model	Treatments	Repeats	Observations per treatment	Status
04-12-08-7802	deepseek/deepseek-chat	lattice, standard	3	27	valid
04-26-46-9672	deepseek/deepseek-chat	lattice, standard	3	27	valid exploratory replicate
04-34-20-1272	deepseek/deepseek-chat	lattice, standard, naive	3	27	primary three-treatment run
04-37-05-9782	gpt-4.1	Lattice, standard	2	18	valid cross-model run
04-26-48-8112	Invalid OpenRouter model identifier	Lattice, standard	2	0 valid	excluded protocol failure

The invalid run is retained because silent omission would bias the reproducibility record. The requested identifier `anthropic/claude-sonnet-4-20250514` was rejected by OpenRouter as not a valid model ID before generation.

Primary analysis set

The three-treatment deepseek run is primary for token economy because it directly compares all three treatments under one model and repeat schedule. The gpt-4.1 run is a cross-model robustness check, not a pooled estimate. The two deepseek lattice/standard runs are exploratory replications and are shown separately because they were not preregistered as a single pooled sample.

Visualizations

The generated report contains:

1. accuracy by task with standard-error caps;
2. lenient accuracy by task;
3. total provider tokens by task;
4. tokens per correct outcome;
5. latency by task;
6. paired efficiency scatter with Lattice-to-Standard arrows.

The manuscript uses the same outcome vocabulary and points to the exact HTML reports in [sec:reproducibility].

Statistical reporting policy

No p-value is treated as a universal claim. We report its model, task battery, repeat count, pairing, effect size, and known prompt-leakage mechanism. A larger study should increase repeats to at least 10, add preregistered tasks, separate answer-availability strata, and use a mixed-effects or randomization analysis appropriate to binary outcomes.

System boundary

```

flowchart TB
    subgraph Repo[Repository under test]
        Src[lib + api + apps + scripts]
        Tests[Vitest suite]
        Data[data/*.json and *.csv]
    end
    subgraph Run[Experiment runner]
        Tasks[Fixed task battery]
        Arms[Alternating treatments]
        Calls[OpenRouter calls]
        Score[Strict + lenient scoring]
        Stats[Paired statistics]
    end
    subgraph Pub[Research outputs]
        SVG[SVG figures]
        HTML[Self-contained HTML]
        MD[Modular Markdown manuscript]
        PDF[Template-rendered PDF]
    end
    Src --> Tasks
    Tests --> Score
    Tasks --> Arms --> Calls --> Score --> Stats
    Stats --> Data
    Stats --> SVG --> HTML
    Data --> MD
    MD --> PDF

```

Figure 3: Mermaid diagram

{=latex}

Results {#sec:results}

Primary three-treatment run

The primary `deepseek/deepseek-chat` three-treatment run (`data/openrouter-lattice-experiment`)

Treatment	Strict accuracy	Lenient accuracy	Mean tokens	Mean latency (ms)
Lattice	0.593 (16/27)	0.593	2,999	4,103
Standard	0.407 (11/27)	0.407	1,036	3,740
Naive corpus	0.444 (12/27)	0.444	22,651	4,528

The Lattice-to-naive token ratio was approximately 0.132, corresponding to 86.759% fewer provider tokens.

The paired Lattice-versus-standard accuracy contrast in this run had $p = 0.096$ and Cohen's $d = 0.37$.

Inferential statistics

The primary paired Lattice-versus-standard statistics are generated from the same per-call results.

Outcome	n	Mean difference	Lattice 95% CI	Standard 95% CI	Paired t	Wilcoxon sign
Strict accuracy	27	0.185	[0.396, 0.789]	[0.211, 0.604]	$t(26)=1.727$, $p=0.096$	W=128, $p=0.096$
Total provider tokens	27	1,963	[1,767, 4,232]	[-47, 2,118]	$t(26)=10.928$, $p<0.001$	
Latency (ms)	27	363	[595, 7,610]	[524, 6,956]	$t(26)=0.577$, $p=0.569$	W=128, $p=0.569$

****Statistics caption.**** Table values are generated from `data/openrouter-lattice-primary-statistics`.

Measurement variables

The manuscript reports strict accuracy, lenient accuracy, total provider tokens, request latency, and provider tokens per request.

Per-task primary results

Task	Lattice acc.	Standard acc.	Naive acc.	Lattice tokens	Standard tokens	Naive tokens
`qa_default_cursor_model`	0.000	0.000	0.000	2,599	81	22,611
`qa_cli_email_env`	0.000	0.000	0.000	2,567	82	22,569
`qa_cli_default_nest`	1.000	0.000	0.000	2,538	79	22,550
`qa_openrouter_header`	1.000	0.000	0.667	2,577	81	22,551
`qa_cli_auth_exit_code`	0.333	1.000	0.333	2,561	77	22,545
`coding_add_top_p_flag`	0.000	0.000	0.000	11,334	8,681	23,411
`reasoning_arithmetic`	1.000	1.000	1.000	373	63	22,531

`reasoning_remainder`	1.000	0.667	1.000	387	94	22,537	
`reasoning_sequence`	1.000	1.000	1.000	2,056	86	22,554	

Cross-model run

The valid `openai/gpt-4.1` run (`data/openrouter-lattice-experiment-2026-08-08T04-37-05-9732`) produced 100% accuracy.

Exploratory replicate

The second valid `deepseek/deepseek-chat` lattice/standard run (`04-26-46-907Z`) produced 100% accuracy.

Mechanism-localized pattern

The lattice advantage is localized. In the primary three-treatment run, Lattice solved `qa\_coding` and `qa\_reasoning` tasks.

Coding failure mode

All treatments failed the coding task. The run-level details show no unified diff, rejected.

Figures and reports

The HTML reports are self-contained and include SVG charts. The manuscript's static figures are available in the supplementary materials.

! [Primary-run strict accuracy by treatment and task] (figures/primary_accuracy.svg) {#fig:accuracy}

! [Primary-run provider tokens by treatment and task] (figures/primary_tokens.svg) {#fig:tokens}

! [Primary-run paired token comparison] (figures/primary_paired.svg) {#fig:paired width=85%}

! [Primary-run latency by treatment and task] (figures/primary_latency.svg) {#fig:latency width=85%}

! [Primary-run lenient accuracy by treatment and task] (figures/primary_lenient_accuracy.svg) {#fig:lenient_accuracy}

! [Primary-run tokens per correct outcome by treatment and task] (figures/primary_tokens_per_correct.svg) {#fig:tokens_per_correct}

Figure captions. Figure [0fig:accuracy] reports strict binary task accuracy; Figure [0fig:tokens] reports provider tokens.

Statistical caution

The $p = 0.042$ result is one exploratory result among multiple runs, models, outcomes, and tasks.

````{=latex}  
\newpage

## Discussion

### Implementation finding

The current Lattice implementation adds a nontrivial system envelope. Against a lean direct prompt, that envelope costs tokens and often latency. Against a large repository dump, it is substantially cheaper. The meaningful engineering target is therefore not “Lattice versus no text”; it is “structured, selective context versus indiscriminate context,” with a direct mode available when no repository grounding is needed.

### Empirical finding

The primary three-treatment run supports a conditional performance story. Lattice was better than standard on two repository QA tasks, tied on reasoning, and tied with both on the coding gate. The gpt-4.1 run reproduced the direction and crossed  $p = 0.05$ , but with only two repeats. The effect is therefore best described as a grounding-sensitive signal.

### Why the naive arm matters

A two-arm Lattice-versus-lean comparison can make a context envelope look inefficient even when the actual product decision is between a compact routing layer and a corpus dump. The naive arm makes that trade-off visible: it consumed roughly 22 times the standard tokens and 7.6 times the Lattice tokens in the primary live run. Its accuracy was not higher than Lattice. This supports using retrieval/pointers as a budget-control mechanism, while leaving open whether the same quality could be obtained with a simpler retrieval prompt.

### Applicability

The result is most applicable to repository questions where the system can reliably expose the relevant fact or file pointer. It is less informative for arithmetic, general knowledge, or tasks requiring iterative tool use. Teams should measure their own model, repository, task mix, and billing unit before adopting a token-efficiency claim.

### Limits of generalization

- **Model:** Two valid models were tested, with unequal repeat counts; one requested model identifier was invalid.
- **Tasks:** Nine hand-authored tasks are not a representative software-engineering benchmark.
- **Scoring:** Exact-match and containment rules are brittle; coding requires a patch protocol that differs from interactive agent use.

- **Prompt leakage:** Lattice context may expose answers. This is part of the treatment’s intended grounding behavior, but it confounds mechanism attribution.
- **Statistics:** Repeats are too few for stable confidence intervals or high-powered model-by-task inference.
- **Costs:** Provider token usage is not a complete cost model; cache discounts, output pricing, retries, and tool calls are outside this study.

## Design improvements

The next preregistered version should:

1. stratify tasks by whether the answer is present in each context;
2. include a retrieval-only arm with the same facts but no nested-agent language;
3. include a context-length-matched standard arm;
4. use at least 10 repeats per model and randomize task order;
5. use tool-enabled coding with an identical patch-and-test harness;
6. report prompt and completion tokens separately;
7. fit a mixed-effects logistic model for accuracy and a robust model for latency/tokens;
8. correct for multiple primary outcomes;
9. validate every model identifier against the live provider catalog before spending calls;
10. preserve raw responses outside the public artifact boundary with access controls if qualitative audit is needed.

## Practical recommendation

Use `nestTopology: "none"` for simple direct questions and use Lattice mode when repository grounding, task decomposition, or cross-band routing is the objective. Do not market the current results as universal accuracy improvement. Market the demonstrated engineering property more narrowly: explicit pointers can make repository facts available while avoiding the cost of sending a large corpus on every request.

## Architecture-to-evidence map

The chart is conceptual: the quantitative coordinates should not be read as an additional measurement. The live tables remain authoritative.

## Honest conclusion

The experiments do not show that Lattice always outperforms a direct prompt. They show a more useful and defensible result: the Lattice envelope is a measurable context cost; it can be much cheaper than a corpus dump; and it can improve repository-answer performance when its grounding exposes the relevant

```
quadrantChart
 title Context cost versus repository grounding
 x-axis Lean context --> Large context
 y-axis Low grounding --> High grounding
 quadrant-1 Expensive but grounded
 quadrant-2 Preferred when selective
 quadrant-3 Cheap but under-informed
 quadrant-4 Avoid indiscriminate dumps
 Standard: [0.18, 0.22]
 Lattice: [0.42, 0.78]
 Naive: [0.93, 0.64]
```

Figure 4: Mermaid diagram

fact. The remaining question is whether a simpler, equally grounded retrieval prompt can achieve the same result at lower envelope cost.

# Reproducibility and audit trail

## Source of truth

The experiment source is:

- `lib/openrouter-experiment.mjs` — task battery, treatment builders, scores, statistics, SVG, and HTML report assembly;
- `scripts/lattice-vs-standard-openrouter-usage.mjs` — provider runner, alternating pairs, result serialization;
- `tests/scripts/openrouter-experiment.test.mjs` — core tests;
- `tests/lib/lattice-prompt.test.mjs` — plain/direct topology tests;
- `docs/OPENROUTER_LATTICE_EXPERIMENT.md` — operational experiment note.

## Result registry

| Artifact                                                                              | Role                                        |
|---------------------------------------------------------------------------------------|---------------------------------------------|
| <code>data/openrouter-lattice-experiment-2026-08-08T04:34:20-127Z.json</code>         | Aggregates results and per-call metadata    |
| <code>data/openrouter-lattice-experiment-2026-08-08T04:34:20-127Z.csv</code>          | Primary results                             |
| <code>reports/openrouter-lattice-experiment-self-2026-08-08T04:34:20-127Z.html</code> | Self-comparison report                      |
| <code>data/openrouter-lattice-experiment-2026-08-08T04:37:05-973Z.json</code>         | Aggregates results and per-call metadata    |
| <code>reports/openrouter-lattice-experiment-2026-08-08T04:37:05-973Z.html</code>      | Comparison report                           |
| <code>data/lattice-vs-standard-comparison-def.json</code>                             | Stochastic structural comparison definition |
| <code>data/lattice-openrouter-experiment-latest.json</code>                           | Latest results                              |

The invalid model run 04-26-48-881Z remains in `data/` as a failure audit and is excluded from valid-run summaries.

## Re-run commands

From the repository root:

```
Protocol only; no provider calls
node scripts/lattice-vs-standard-openrouter-usage.mjs --dry-run

Primary three-treatment design
LATTICE_BENCH_TREATMENTS=lattice,standard,naive \
LATTICE_BENCH_REPEATS=3 \
LATTICE_OPENROUTER_MODEL=deepseek/deepseek-chat \
OPENROUTER_API_KEY='[REDACTED]' \
npm run experiment:openrouter

Cross-model direct comparison
```



```
LATTICE_BENCH_TREATMENTS=lattice,standard \
LATTICE_BENCH_REPEATS=10 \
LATTICE_OPENROUTER_MODEL=openai/gpt-4.1 \
OPENROUTER_API_KEY='[REDACTED]' \
npm run experiment:openrouter
```

The placeholder is intentional. Never put a live key in manuscript source, JSON, CSV, HTML, logs, or shell history. The runner reads the key from the environment and strips raw response text from committed result JSON.

## Verification gates

The implementation was verified with:

```
node --check lib/openrouter-experiment.mjs
node --check scripts/lattice-vs-standard-openrouter-usage.mjs
npm test # 8 files, 80 tests
npm run compare:lattice
npm run test:lattice-floors
git diff --check
```

## Rendering with the template infrastructure

The template renderer expects a project manuscript directory containing ordered Markdown sections, a `config.yaml`, optional `preamble.md`, and a bibliography. This working project is not a Python project with `src/` and `tests/` in the template's usual sense, so the manuscript can be rendered through the renderer's direct configuration path or by placing a temporary project adapter under `projects/working`.

Recommended render setup from the template root:

```
cd /Users/4d/Documents/GitHub/template
MANUSCRIPT_DIR=/Users/4d/Documents/GitHub/template/projects/working/psw.vibelandia.sing13/d
OUTPUT_DIR=/Users/4d/Documents/GitHub/template/projects/working/psw.vibelandia.sing13/docs/n
uv run python -m infrastructure.rendering --help
```

Use the renderer's PDF command exposed by the installed infrastructure version. The manuscript directory contains no generated PDF as source; output is disposable and should remain outside the source sections.

## Provenance

Each result JSON records model, repeat count, treatment set, task IDs, aggregate outcomes, and per-call metadata. The visual report is generated from the same in-memory aggregate. This gives a direct data-to-figure binding, while the manuscript tables are a human-readable frozen summary of the recorded artifact.

## **Security boundary**

Credentials are runtime inputs only. The manuscript contains no API keys, response bodies, authorization headers, or private allowlist data. The invalid model identifier is safe to report because it is a public model-name string and the failure occurred before generation.

## Document map and modularization

### Manuscript modules

| Module                | Purpose                                                 |
|-----------------------|---------------------------------------------------------|
| 00_abstract.md        | Abstract, contributions, honesty boundary               |
| 01_introduction.md    | Background, research questions, architecture            |
| 02_formalism.md       | Treatment model, outcomes, paired statistics, threats   |
| 03_methods.md         | Task battery, protocol, runs, figures, system boundary  |
| 04_results.md         | Primary, cross-model, per-task, and failure results     |
| 05_discussion.md      | Mechanisms, applicability, limitations, recommendations |
| 06_reproducibility.md | Artifact registry, commands, rendering, security        |
| 07_document_map.md    | This map and document organization contract             |
| 99_references.md      | Bibliography and repository references                  |
| config.yaml           | Renderer metadata and output format configuration       |
| preamble.md           | LaTeX packages and layout additions                     |
| SYNTAX.md             | Local Markdown/Pandoc syntax and label registry         |

### Documents directory organization

The legacy docs/ corpus contains many historically named whitepapers and technical notes. To preserve links and historical filenames, this pass does not rename or move legacy files. Instead, organization is provided through stable index documents and purpose-based collections:

- docs/README.md — top-level navigation and honesty boundary;
- docs/INDEX\_BY\_TOPIC.md — topic map for architecture, protocols, research, product/UI, and operations;
- docs/INDEX\_BY\_LIFECYCLE.md — seed/origin, implementation, evidence, and archive map;
- docs/OPENROUTER\_LATTICE\_EXPERIMENT.md — operational experiment protocol;
- docs/manuscript/ — renderable research manuscript source;
- reports/ — generated HTML/SVG experiment reports;

- **data/** — generated JSON/CSV evidence and historical comparison receipts.

Generated outputs are not copied into source sections. The manuscript points to report artifacts and reproduces key figures from the same aggregated data.

## Seed:Edge document contract

Every document should identify:

1. **Seed:** the originating question, protocol, or formal claim;
2. **Edge:** the implementation, user experience, or empirical observation;
3. **Boundary:** what the document does not establish;
4. **Evidence:** source files, runs, tests, or citations;
5. **Lifecycle:** current, exploratory, superseded, or archival status.

This contract keeps conceptual whitepapers, implementation notes, and empirical reports from being silently treated as equivalent evidence.

## Proposed future physical taxonomy

A future migration may place files into these directories after a link audit:

|               |                                            |
|---------------|--------------------------------------------|
| docs/         |                                            |
| architecture/ | system and protocol architecture           |
| protocols/    | operational protocol specifications        |
| research/     | hypotheses, methods, and empirical reports |
| product/      | UI, catalog, and user-facing behavior      |
| operations/   | deployment, access, and maintenance        |
| archive/      | superseded or historical documents         |

This pass intentionally leaves the physical legacy corpus stable. The index-first approach is reversible and avoids breaking external references while the repository is actively evolving.

## Rendering contract

The renderer consumes ordered numeric Markdown modules. Filenames are part of the composition protocol: lower numbers establish context and definitions; **99\_references.md** terminates the body with references. Figures use relative paths under **docs/manuscript/figures/** and are generated before rendering.

## References

This manuscript is primarily a software and empirical artifact report. External claims should be added here only when they are used in the prose and can be verified independently.

## Repository sources

- `lib/openrouter-experiment.mjs` — experiment core, treatment constructors, scoring, paired statistics, and SVG/HTML report functions.
- `scripts/lattice-vs-standard-openrouter-usage.mjs` — live OpenRouter runner and result serializer.
- `lib/lattice-prompt.mjs` — Lattice prompt assembly and topology normalization.
- `lib/lattice-engine.mjs` — token accounting and structural execution envelopes.
- `tests/scripts/openrouter-experiment.test.mjs` — experiment-core tests.
- `tests/lib/lattice-prompt.test.mjs` — plain/direct topology tests.
- `docs/OPENROUTER_LATTICE_EXPERIMENT.md` — operational protocol note.

## Run artifacts

- Primary three-treatment run: `data/openrouter-lattice-experiment-2026-08-08T04-34-20-127Z.js`
- Primary visual report: `reports/openrouter-lattice-experiment-2026-08-08T04-34-20-127Z.html`.
- Valid cross-model run: `data/openrouter-lattice-experiment-2026-08-08T04-37-05-973Z.json`.
- Structural comparison receipt: `data/lattice-vs-standard-comparison.json`.

## Citation policy

The current manuscript deliberately avoids unverifiable external citations. When related-work citations are added, use Pandoc citation syntax and add complete entries to `references.bib`; do not add citations merely to decorate the background.